

M9 — What this block gets you

- ✓ The 'traditional data warehouse challenges' on the next two slides are your SQL Server pain points—read them that way
- ✓ BigQuery fundamentals— serverless, storage/compute separation, slots and shuffle in plain terms
- ✓ Partitioning and clustering— your partition functions and clustered indexes, minus the maintenance jobs
- ✓ Iceberg partitioning and predicate pushdown— open metadata pruning files that live in your own bucket
- ✓ Both are a preview— the 3-day Data Warehousing with BigQuery course goes deep; today is the decision rules

Late morning into early afternoon

11:35–12:00 M9 — Modernizing DWs with BigQuery & BigLake (opens with your pain points)

12:00–13:00 Lunch

13:00–13:45 Lab: Querying External Data and Iceberg Tables — your Bronze pattern, hands-on

LAB

13:45–14:00 Iceberg deep-dive — what's under the table you just built

DEMO

14:00 M10 — governance + medallion: your Bronze/Silver/Gold in Google's own deck

Traditional data warehouse challenges (1 of 2)

Cymbal's data architects face significant challenges with traditional data warehouses:

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

02

Difficulty handling unexpected traffic spikes.

03

Manual provisioning of hardware, software installation, and patching.

04

Rebalancing data manually as it grows.

Traditional data warehouse challenges (2 of 2)

Operational overhead is **expensive** and **slows down** the ability of the analytics teams to get **timely insights**.

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

03

Manual provisioning of hardware, software installation, and patching.

02

Difficulty handling unexpected traffic spikes.

04

Rebalancing data manually as it grows.

Those five challenges are yours — this is the delete key

Each one is an infrastructure problem — the fix is to remove the infrastructure.

- Capacity planning for peak events— serverless slots, allocated per query and released when it finishes
- Sized for peak, idle the rest of the day— storage and compute scale and bill independently
- Procurement, installation, patching weekends— fully managed; that M&O; line item disappears
- Rebalancing data as it grows— storage scales automatically; no new drives, no filegroup shuffles
- The nightly window sized the whole box— this engine was built for exactly the scans you avoid
- Slow answers for analysts— thousands of slots per query: a wider machine, not a faster one

BigQuery

Enterprise-grade, fully managed,
serverless data warehouse.



BigQuery fundamentals

Fully managed

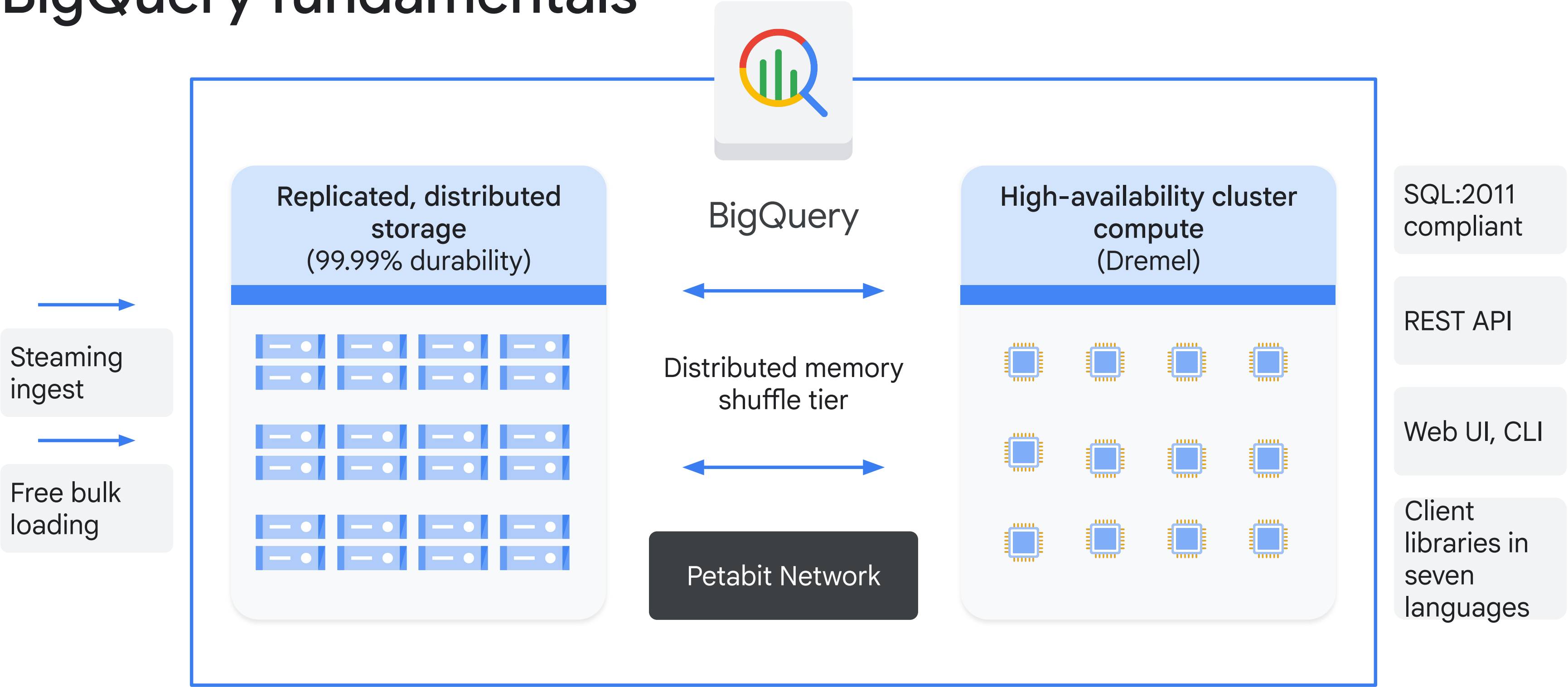
The infrastructure (such as the hardware, the networking, and the low-level software) is all handled by Google. Your team doesn't need to worry about patches, updates, or hardware failures.

Serverless

Serverless takes things a step further. You don't have to provision or manage any servers at all. You simply load your data and start querying. BigQuery automatically allocates the necessary resources to run your queries and scales them up or down based on the complexity of your request.

The magic behind BigQuery is its architecture. BigQuery **separates storage from compute.**

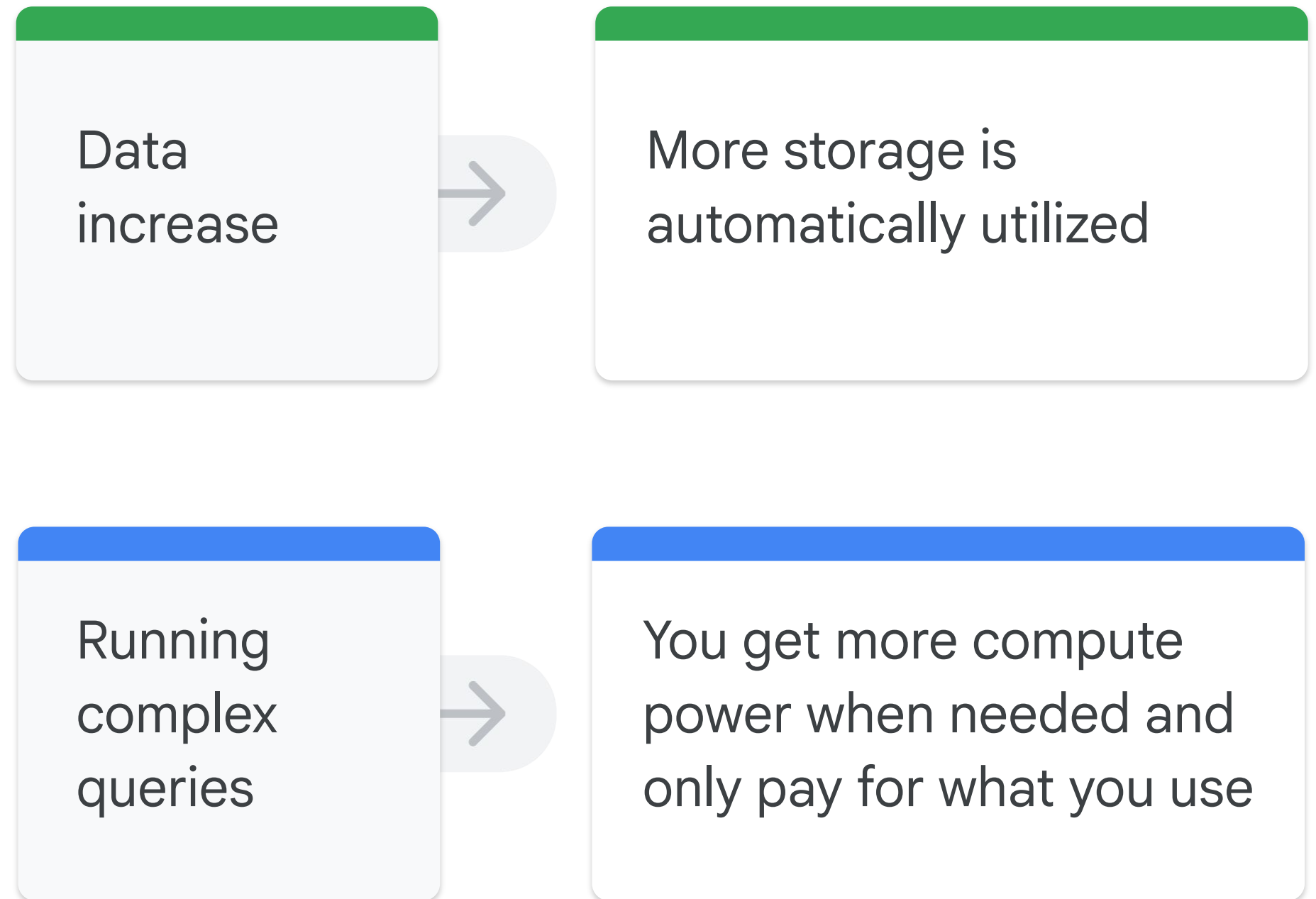
BigQuery fundamentals



BigQuery fundamentals

Because storage and compute are separate, they can scale independently.

This separation is a game changer for managing costs and ensuring performance.



BigQuery achieves its speed through two core concepts: **slots and shuffle**.

What is a slot?

A small, self-contained unit of computational power that includes **CPU**, **RAM**, and **network bandwidth**.



Slots

01

BigQuery's Dremel engine assigns thousands of slots to a query job.

02

Each slot processes a small piece of data simultaneously.

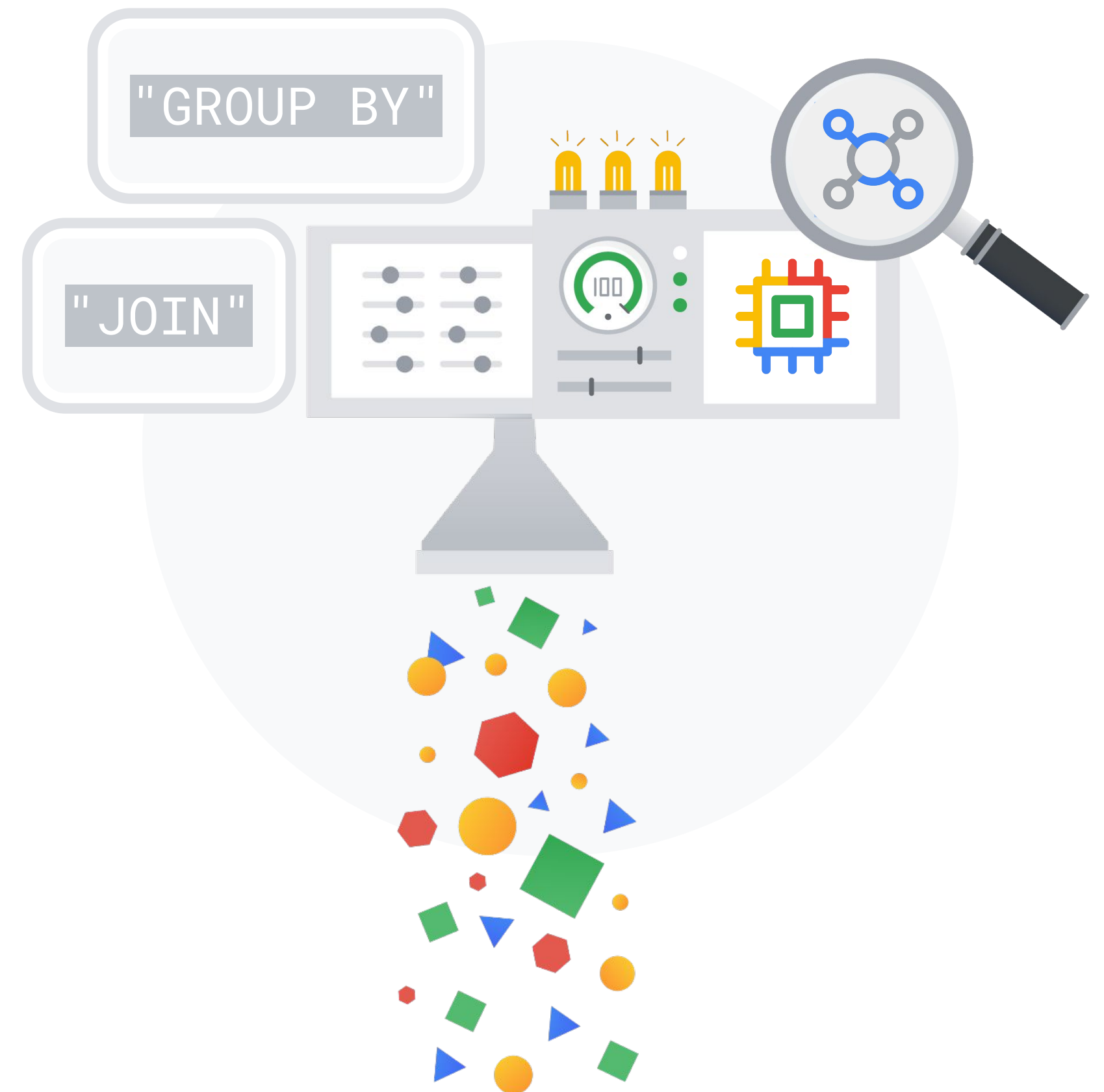
03

This massively parallel processing enables BigQuery to scan terabytes of data quickly.



What is shuffle?

The process of redistributing the intermediate data that the slots have processed.



Shuffle

01

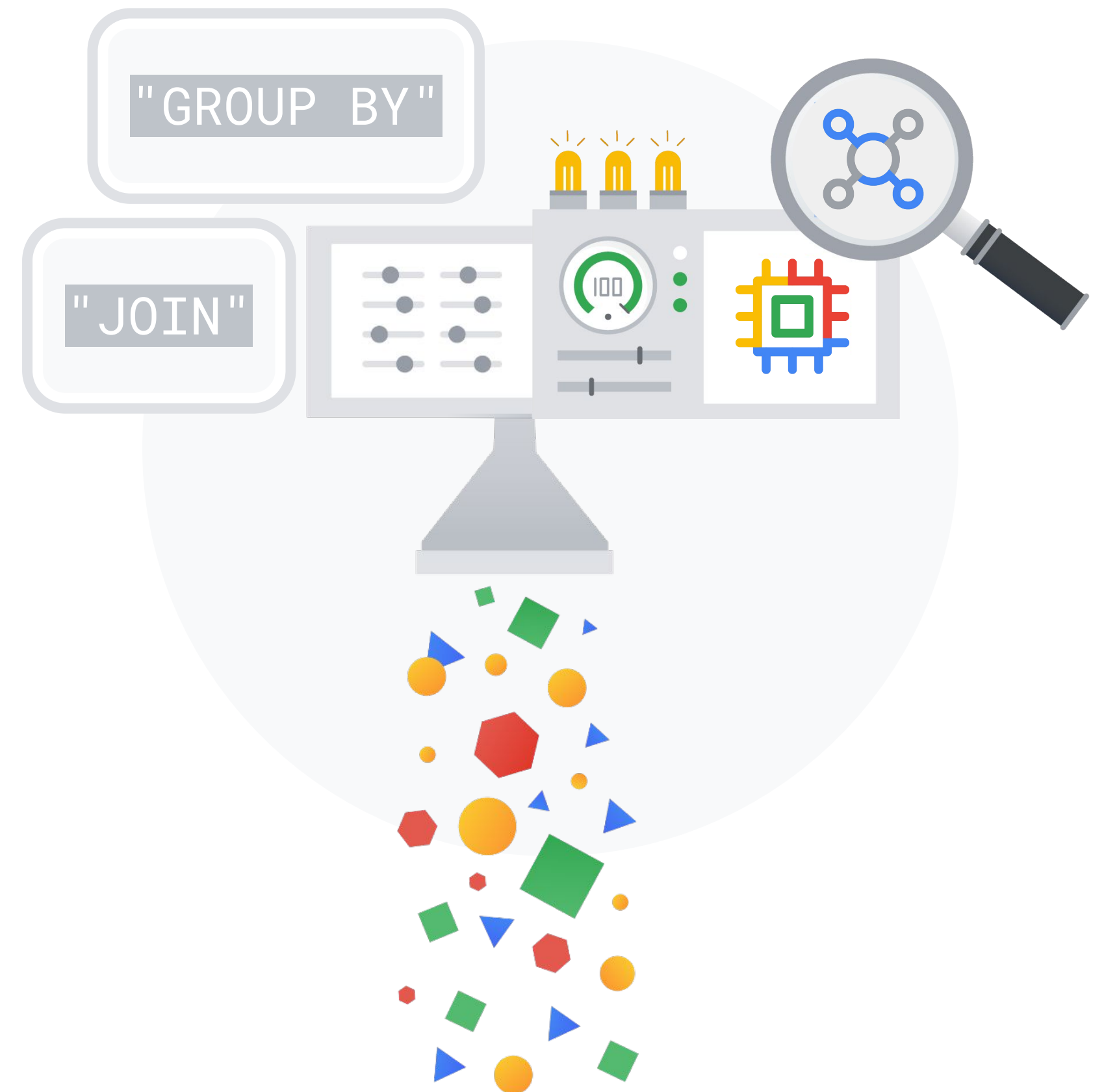
Gathers and reorganizes data.

02

Sends it to the next set of slots for further processing, such as aggregation or joining.

03

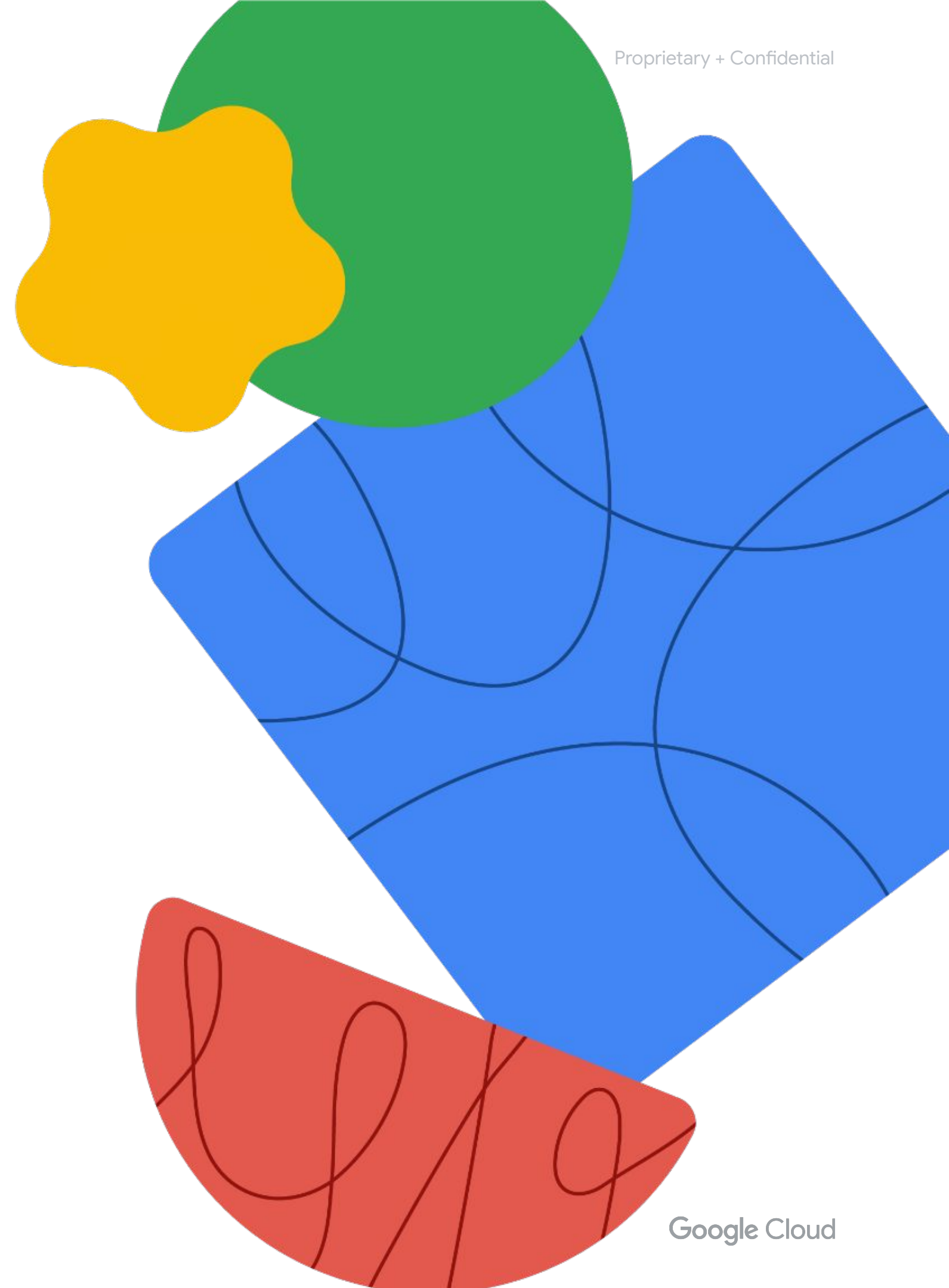
Executes complex analytical queries efficiently at a massive scale.



BigQuery's compute and storage separation enables versatile **multisource data querying.**

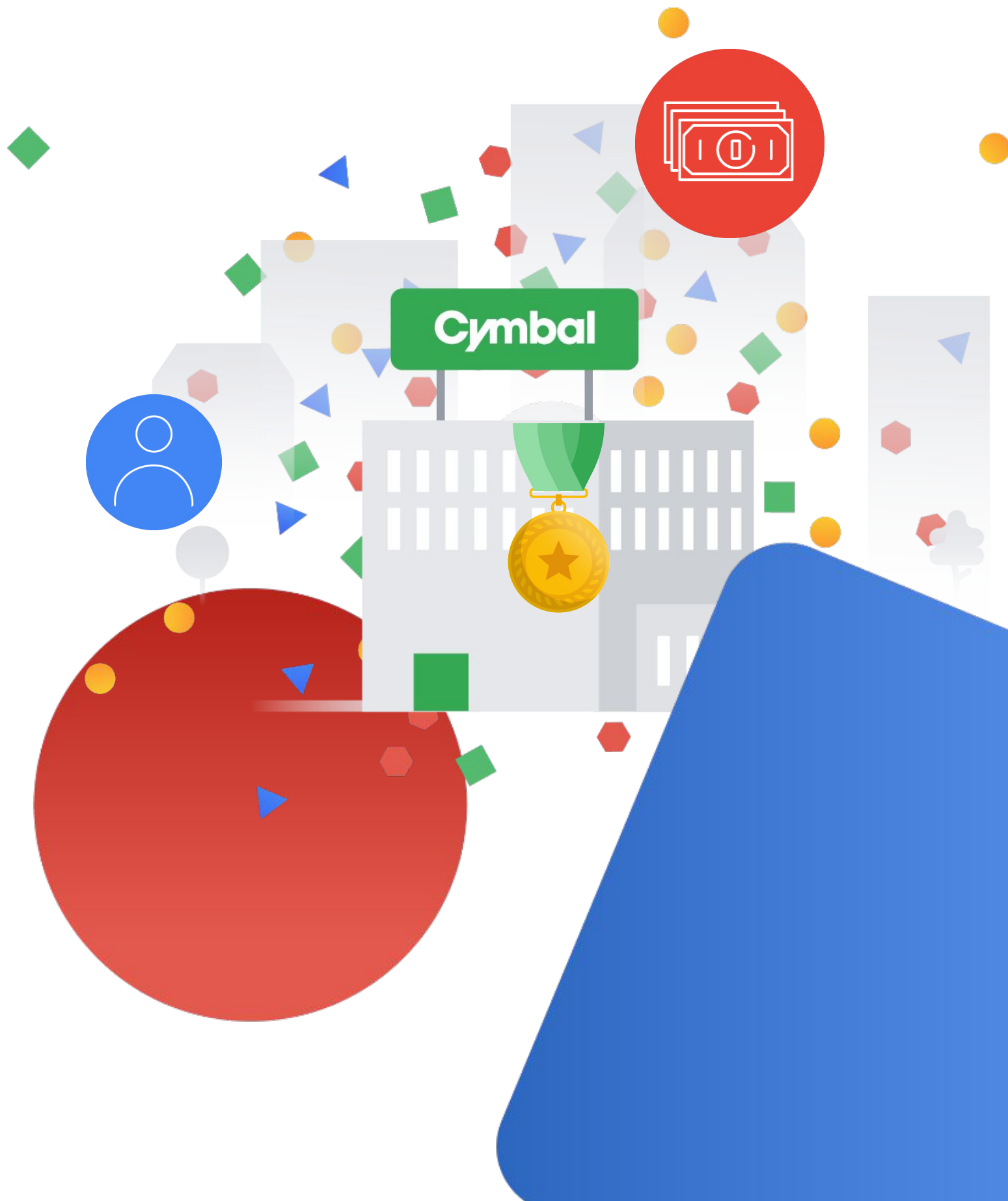
Partitioning and clustering

Two powerful concepts you can use to allow BigQuery to process your data efficiently and cost-effectively.



“

Running queries that scan every single piece of data can become slow and expensive, even for BigQuery. You can optimize this with **partitioning and clustering**.



Partitioning

Partitioning is like organizing data into separate drawers for year, month, and day. In BigQuery, you can partition a table based on a date or an integer column.



Example: Sales data table

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57
2025-08-03	CUST003	Shoes	413
2025-08-03	CUST001	Consoles	347

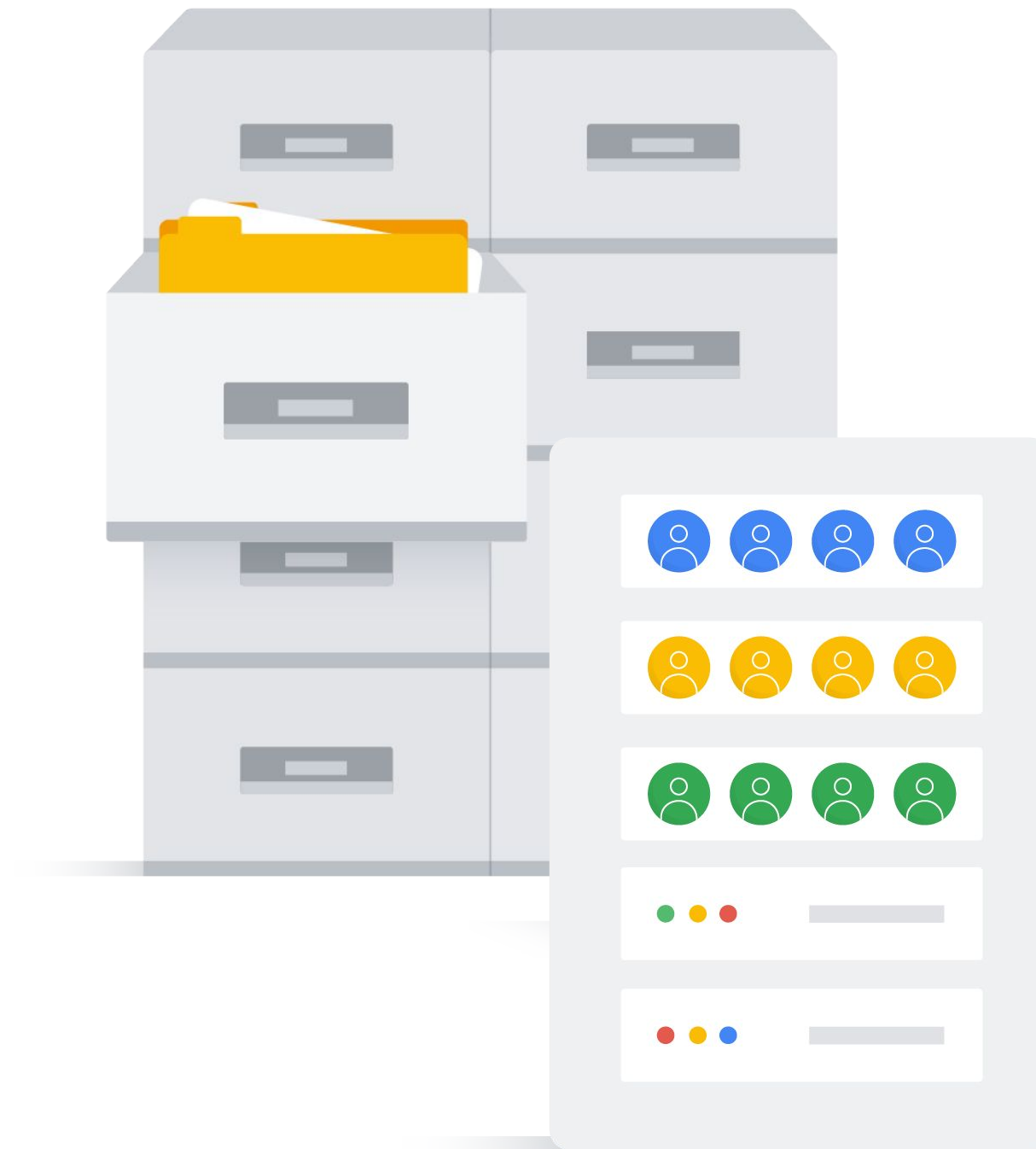
Partitioned by **transaction_date**

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186

transaction_date	customer_id	product_category	sales_amount
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57

Clustering

Clustering sorts the data within each partition. Think of it as organizing the files within each drawer of your filing cabinet alphabetically by customer name.



Clustering example

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-01	CUST003	Headsets	465
2025-08-01	CUST005	Headsets	57
2025-08-01	CUST003	Shoes	413
2025-08-01	CUST001	Consoles	347

Query example with partition + clustering

```
SELECT *  
FROM cymbal_sales  
WHERE transaction_date = '2025-08-01'  
      AND customer_id = 'CUST002';
```


Partitioning & clustering — your tuning toolkit, translated

Same disciplines you use today, declared at CREATE TABLE time — then never maintained again.

- Partition functions and sliding windows—
partitioned tables; pruning is automatic on a date filter
- Clustered indexes and index tuning—
clustering columns: sort order inside storage blocks, up to four
- Index rebuilds in the maintenance plan—
automatic background reclustering; there is no job to write
- No retroactive ALTER— repartition by rebuilding with CREATE TABLE AS SELECT
- Decision rule— partition on the date your reports filter; cluster on your top filter/join columns
- Aim for ~1–10 GB per partition— daily by default; tiny partitions slow metadata, not queries

Preview, not the full story — the 3-day Data Warehousing with BigQuery course being scheduled goes deep on both, hands-on.

Partitioning in Iceberg

01

Leveraging existing iceberg structures

Intelligently leverages the existing structure defined in the Apache Iceberg table's own metadata.

02

Defining partitions

With partitioning, the partitions are typically defined and written by a data processing engine, such as Apache Spark.

03

Data organization

The Apache Iceberg metadata meticulously tracks which specific data files in Cloud Storage belong to which date.

Query filtering example

```
WHERE transaction_date = '2025-08-15'
```

01

BigQuery (via Lakehouse) reads this metadata and identifies the exact set of files corresponding to that date and prunes away all others.

02

Ignores irrelevant data, drastically reducing the amount of data scanned, which lowers costs and accelerates query performance.

Clustering in Apache Iceberg

01

Clustering in BigQuery is mirrored by data sorting and file-level statistics in Apache Iceberg.

02

The data within Apache Iceberg's Parquet or ORC files is often sorted by specific columns, such as `customer_id`.

03

The Apache Iceberg metadata stores statistics, such as the minimum and maximum `customer_id` values, for each individual data file.

Predicate pushdown

01

If your BigQuery query filters for a specific customer, the query planner consults these statistics.

02

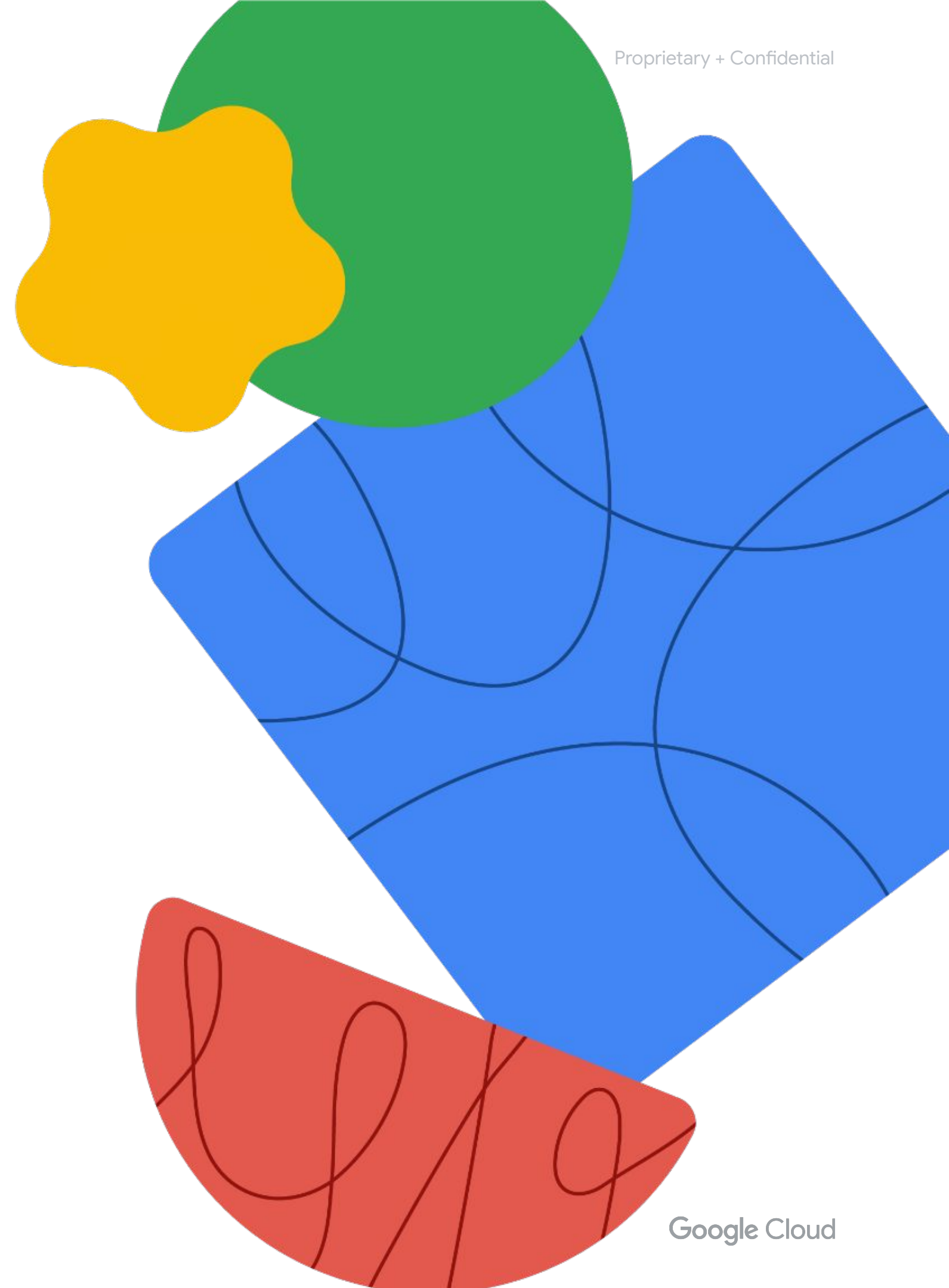
Skips reading any files whose min/max range doesn't contain the requested `customer_id`.

03

Predicate pushdown provides a finer-grained level of data pruning, ensuring maximum query efficiency.

Why it matters

Partitioning and clustering together give BigQuery fine-grained control over how your data is queried, leading to significant performance improvements and cost savings, especially as your data grows.



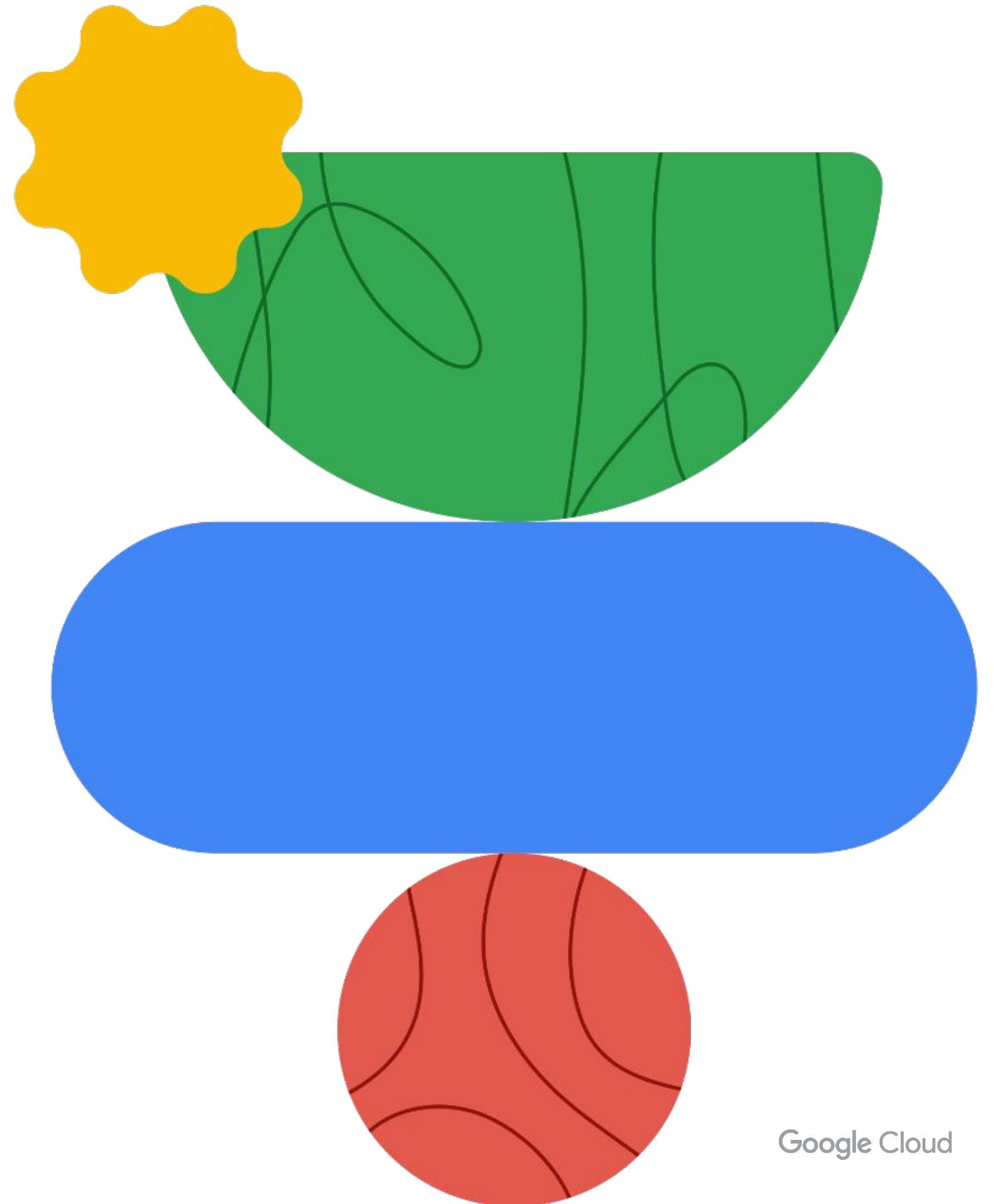
BigQuery uses metadata to skip unnecessary data, so **queries run faster** and cost less.

Iceberg partitioning — the same pruning, on files you own

Why your Bronze layer cares: filtering works before anything is ever loaded.

- Iceberg metadata tracks which Parquet files belong to which partition— open metadata, in your bucket
- Per-file min/max statistics let the planner skip files that can't contain your value— predicate pushdown
- Same discipline as partition elimination and index seeks— but any Iceberg-speaking engine can read it
- This is the answer to 'will BigQuery hold our data hostage?'— open files plus open metadata
- Also a preview— Iceberg internals get the deep, hands-on treatment in the Data Warehousing course

**Valuable data exists in
many places and
formats**



Data warehouse vs. data lake

Traditional data storage

- Data warehouse: Ideal for structured, organized data used for business insights.
- Data lake: A cost-effective place for huge amounts of raw data in any format.

Challenges of separation

- Data silos: Hard to analyze different data types together.
- Complex extract, transform, load (ETL): Required moving and duplicating data, increasing costs and delays.
- Governance Issues: Managing security and access across two systems was complicated.

A data lakehouse **solves these challenges.**

Data lakehouse architecture

01

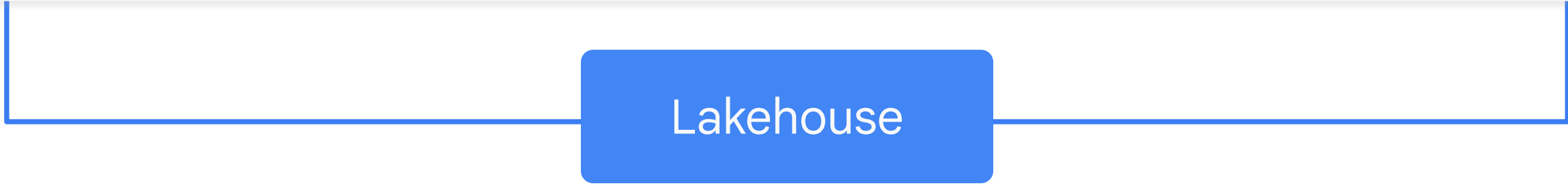
Combines low-cost, flexible data lake storage.

02

Offers powerful querying, transaction management, and governance features of a data warehouse.

03

Delivers a single, unified system for modern data architecture.

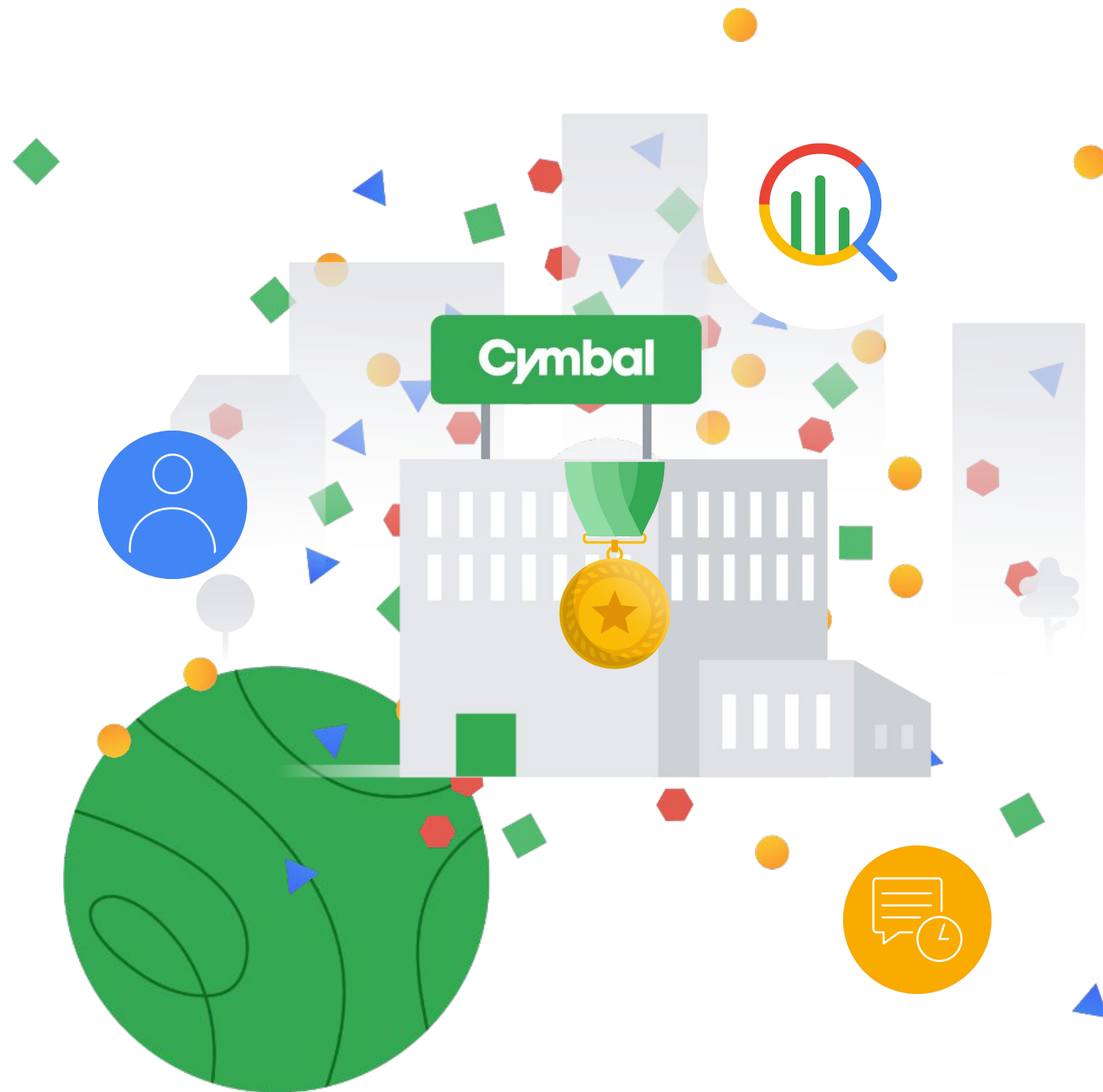


Lakehouse

Lakehouse and external tables

- 01 Lakehouse extends BigQuery.
- 02 Create Lakehouse tables over data in object stores, such as Google Cloud Storage.
- 03 Reference the source data files in your data lake, and query them.





How can Cymbal use Lakehouse external tables?

- Cymbal's data science team uses Cloud Storage to store raw web server log files in JSON format.
- Analyzing this clickstream data used to require a complex data pipeline to parse and load it into BigQuery.

Lakehouse simplifies this process.

Lakehouse external table

01

They create a Lakehouse table directly over the JSON files in Cloud Storage.

02

Use the familiar BigQuery SQL interface to query clickstream data instantly.

03

Join this external table with a native BigQuery sales table, to discover correlations between website behavior and purchasing habits.

Lab: Introduction to querying Apache Iceberg tables with BigQuery

Overview

Explore how BigQuery's Lakehouse tables can query data in open-source formats directly within Cloud Storage.



What you'll do

- Create a Lakehouse table that points to an existing Apache Iceberg table.
- Analyze data without moving or duplicating it.

Lab: Querying External Data and Iceberg Tables

45 minutes · Qwiklabs seat — BigQuery console

1. Create a Lakehouse (Iceberg) table pointing at the existing Apache Iceberg data in Cloud Storage
2. Suffix your table name with your initials — 16 identical DDL statements in one dataset will collide
3. Query it: filter and aggregate — plain SQL over files that were never loaded anywhere
4. Watch the dialect: backticks not brackets, LIMIT not TOP, dataset in the fully qualified name
5. Notice the DDL is metadata-only and returns instantly — nothing moved, nothing copied

Why it matters — this is your Bronze pattern, hands-on — raw data stays in open formats in Cloud Storage and is queryable from minute one. Bronze is not a landing zone you copy out of.

Iceberg deep-dive — what's under the table you just built

15 minutes · Instructor demo — BigQuery console (from Jeff's Iceberg tutorial, gcp-demos)

1. Frame the one idea that matters: three Iceberg flavors differ by who owns the metadata and who can write
2. Debrief the lab: what your Iceberg table has that this morning's external table didn't — DML, time travel, snapshots
3. Upgrade the morning's ext_part external table to a BigLake table — same files, richer metadata
4. Dwell on WITH CONNECTION — the connection's service account reads the bucket; when it fails, it's that IAM grant
5. Payoff: column-level security and row access policies on files that never left the lake

Why it matters — files became a table this morning, got ACID semantics in the lab, and get governance now — three rungs of one ladder, and the rungs are the Bronze section of your architecture doc. Fallback if the tutorial is still under construction: extended lab debrief plus this BigLake demo.

M9 — What you now have

- ✓ Your SQL Server pain points named on Google's own slides— and the answer: delete the infrastructure layer
- ✓ Slots and shuffle in plain terms— a wider machine, not a faster one; 'bytes shuffled' is where slow queries hide
- ✓ Partitioning and clustering decision rules— flagged as a preview of the Data Warehousing course's deep dive
- ✓ Hands-on Bronze: an Iceberg/Lakehouse table over files in Cloud Storage, queried without loading anything
- ✓ Next: M10 at 14:00— Google's own Bronze/Silver/Gold zone slides, held up next to your architecture doc